

Grafana Monitoring AWS EC2 Instance Metrics: Complete Guide & Step-by-Step



Created by **Kranthi Putti**

Join our group <https://www.linkedin.com/groups/14483928/> for amazing free resources

Grafana Monitoring for AWS EC2 Server

This document provides a step-by-step guide to setting up **Grafana** for monitoring an AWS EC2 instance. It covers installation, configuration, and visualization of system metrics. The guide ensures real-time monitoring of CPU, memory, disk usage, and network activity to optimize server performance and detect issues proactively.

Step 1: Create an IAM User for Grafana

- Navigate to **AWS IAM Console** → **Users** → **Create User**.
- Assign the following AWS-managed policies:
 - **CloudWatchReadOnlyAccess**
 - **CloudWatchLogsReadOnlyAccess**
- Generate **Access Key** and **Secret Access Key**
 - Store the credentials securely

The screenshot shows the 'Review and create' step in the AWS IAM console. On the left, a progress bar indicates four steps: Step 1 (Specify user details), Step 2 (Set permissions), Step 3 (Review and create - currently active), and Step 4 (Retrieve password). The main content area is titled 'Review and create' and includes a sub-header 'Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.' Below this, there are three sections: 'User details' showing 'User name' as 'kranthi', 'Console password type' as 'Custom password', and 'Require password reset' as 'No'; 'Permissions summary' showing two AWS managed policies, 'CloudWatchLogsReadOnlyAccess' and 'CloudWatchReadOnlyAccess', both used as 'Permissions policy'; and 'Tags - optional' with a note that tags are key-value pairs for resource identification.

The screenshot shows the 'Create access key' dialog box. It has a title 'Create access key' and a subtitle 'Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to'. There are five radio button options: 'Local code' (You plan to use this access key to enable application code in a local development environment to access your AWS account.), 'Application running on an AWS compute service' (You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.), 'Third-party service' (You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.), 'Application running outside AWS' (selected; You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.), and 'Other' (Your use case is not listed here.).

The screenshot shows the AWS IAM console for user 'kranthi'. The left sidebar shows the navigation menu with 'Identity and Access Management (IAM)' selected. The main content area has a top bar with 'Assign MFA device'. Below it, the 'Access keys (0)' section shows a 'Create access key' button and text: 'Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. Learn more'. Below this, it says 'No access keys. As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short term credentials. Learn more' and another 'Create access key' button. The 'SSH public keys for AWS CodeCommit (0)' section has an 'Upload SSH public key' button and text: 'User SSH public keys to authenticate access to AWS CodeCommit repositories. You can have a maximum of five SSH public keys (active or inactive) at a time. Learn more'. At the bottom, there is a table with columns 'SSH Key ID', 'Uploaded', and 'Status'.

Step 2: Install Grafana

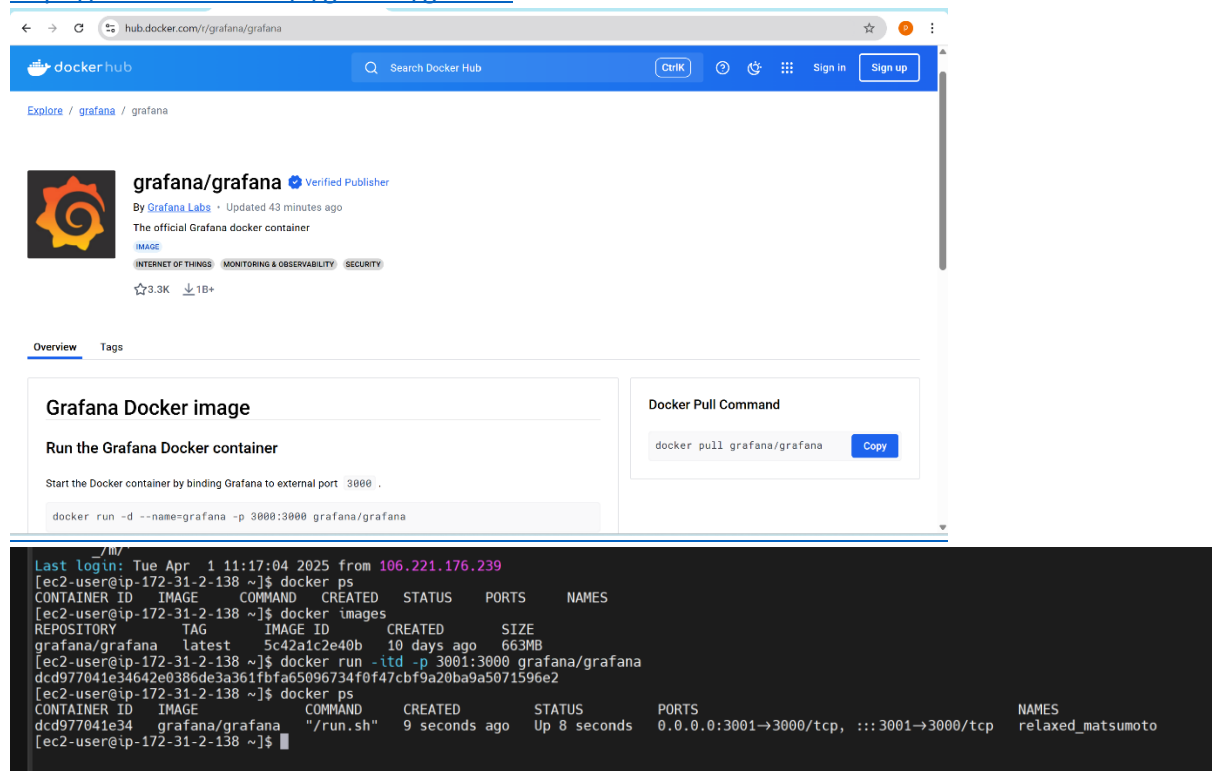
Choose one of the installation methods:

Official Package – Install using the official repository <https://grafana.com/docs/grafana/latest/setup-grafana/installation/>

Grafana Cloud – Sign up at Grafana Cloud <https://grafana.com/products/cloud/>

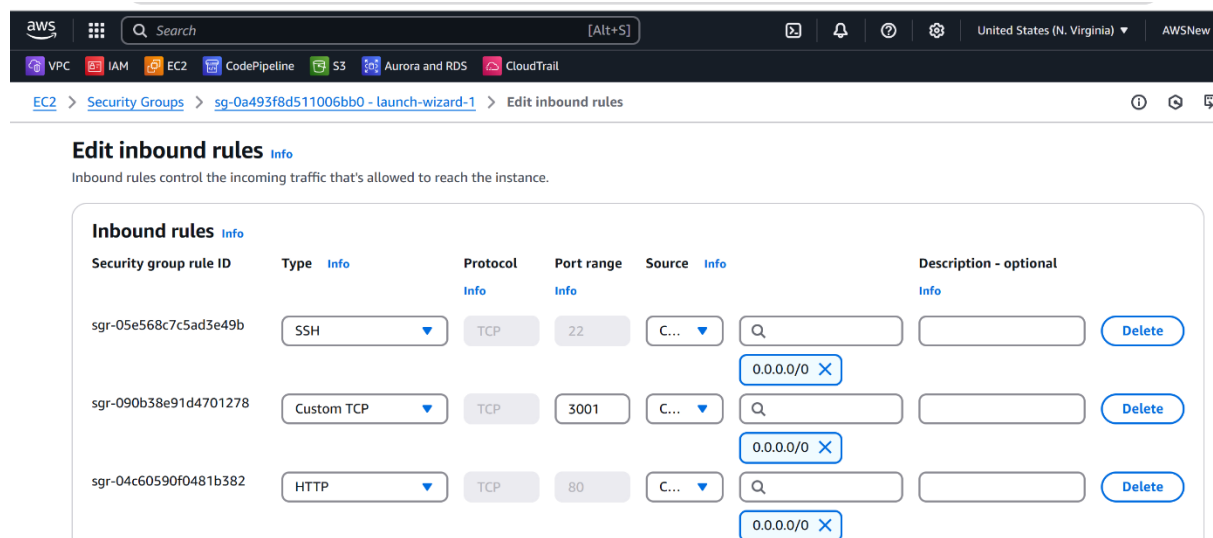
Docker – Deploy Grafana using: `docker run -d -p 3000:3000 --name=grafana grafana/grafana`

<https://hub.docker.com/r/grafana/grafana>



The screenshot shows the Docker Hub page for the `grafana/grafana` image. The page includes the Docker Hub logo, search bar, and navigation links. The main content area displays the `grafana/grafana` image, which is a verified publisher. It shows the image's details, including the repository, tags, and a pull command: `docker pull grafana/grafana`. Below this, there is a section titled "Grafana Docker image" with instructions on how to run the container. A terminal window at the bottom shows the command `docker run -d --name=grafana -p 3000:3000 grafana/grafana` being executed, and the output shows the container starting successfully.

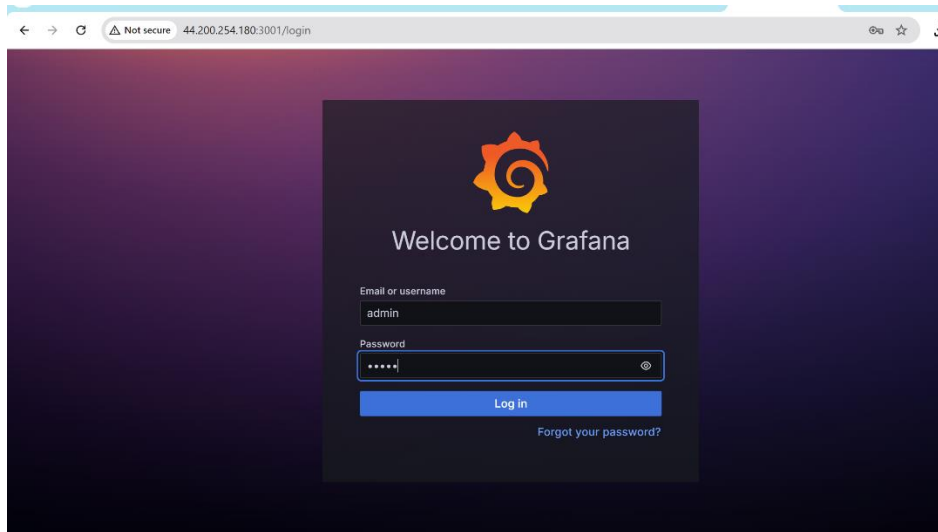
- Allow **TCP port 3000** in the EC2 security group for Grafana access.



The screenshot shows the AWS Management Console interface. The top navigation bar includes the AWS logo, search bar, and various service icons. The main content area displays the "Edit inbound rules" page for a security group. The page shows a table of inbound rules with columns for Security group rule ID, Type, Protocol, Port range, Source, and Description - optional. The table lists three rules: one for SSH (port 22), one for Custom TCP (port 3000), and one for HTTP (port 80). Each rule has a "Delete" button next to it.

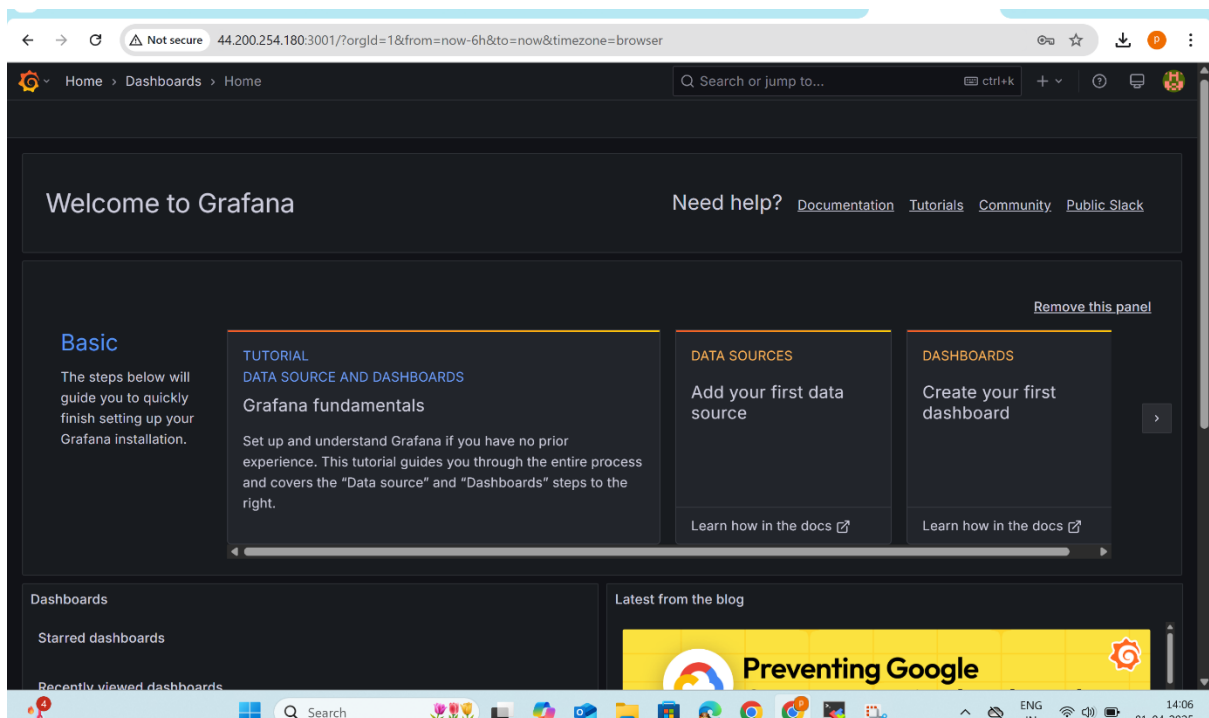
Step 3: Sign in to Grafana

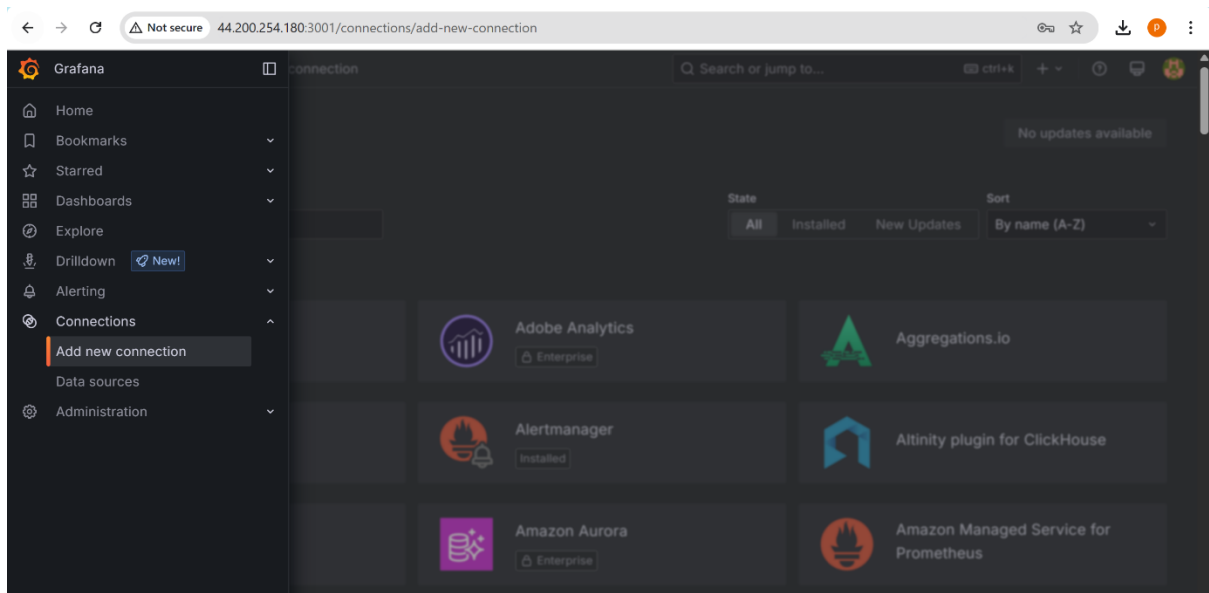
Log in with **Username: admin** and **Password: admin**, then set a new custom password.



Step 4: Add Data Source

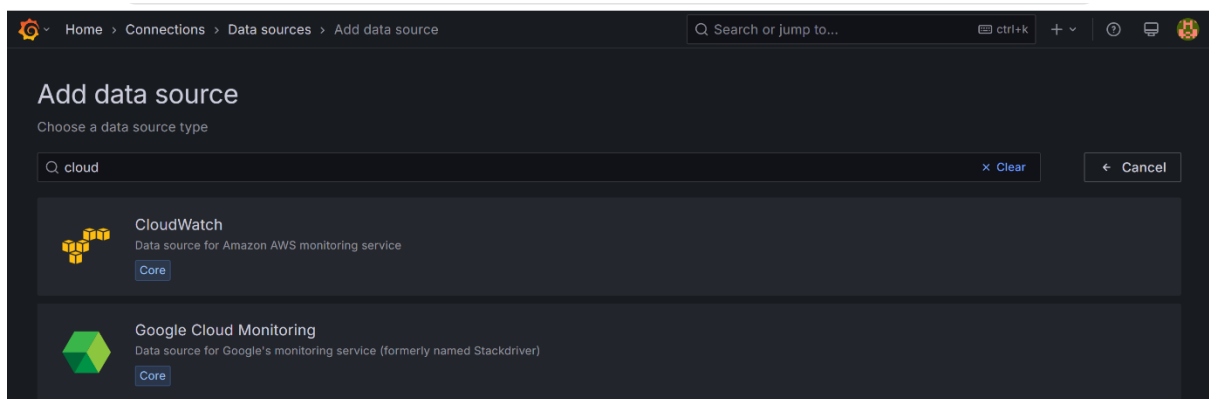
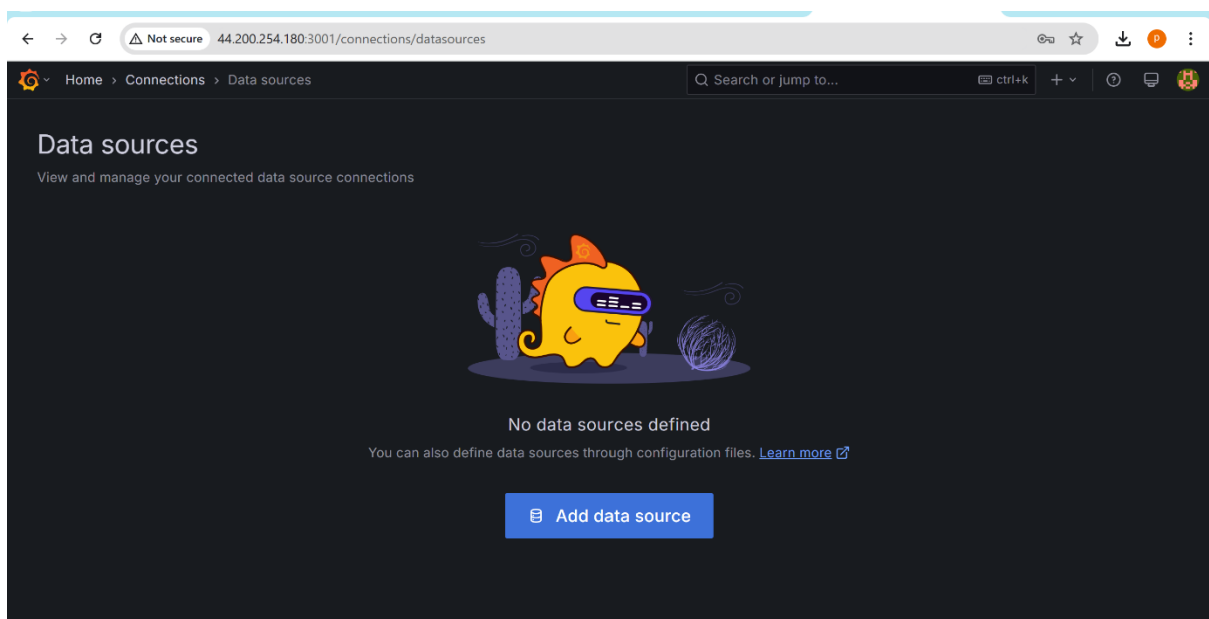
Go to **Connections** → **Data Sources**, select **CloudWatch**, and configure it for your AWS EC2 server.





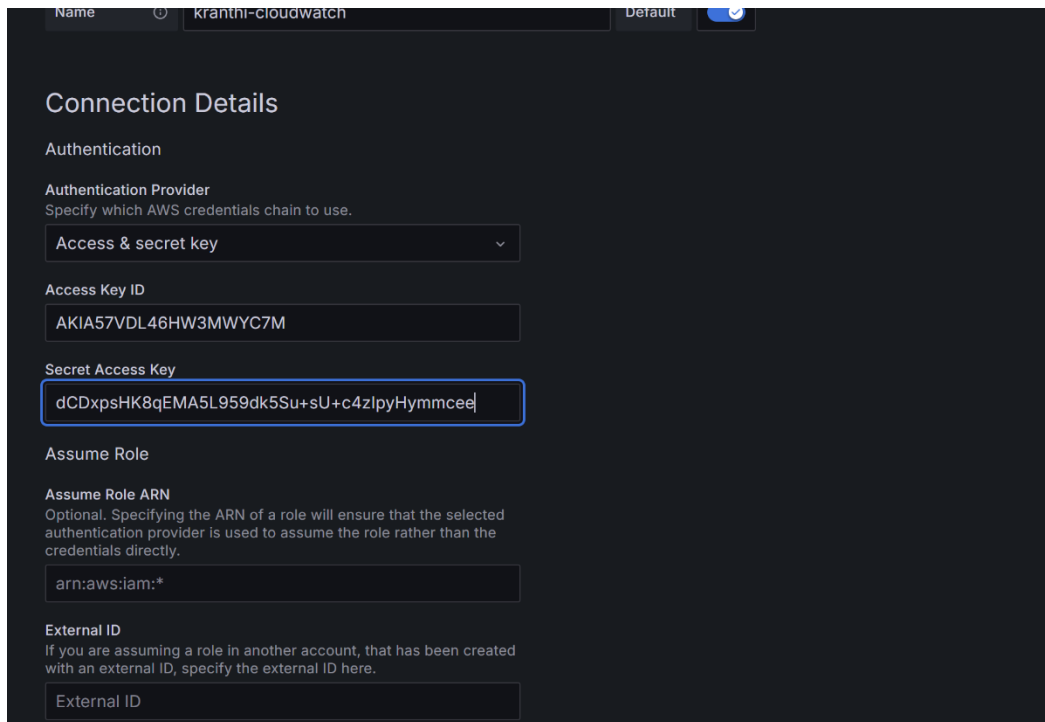
Step 5: Select CloudWatch as Data Source

Click **Add data source**, search for **CloudWatch**, and select it.



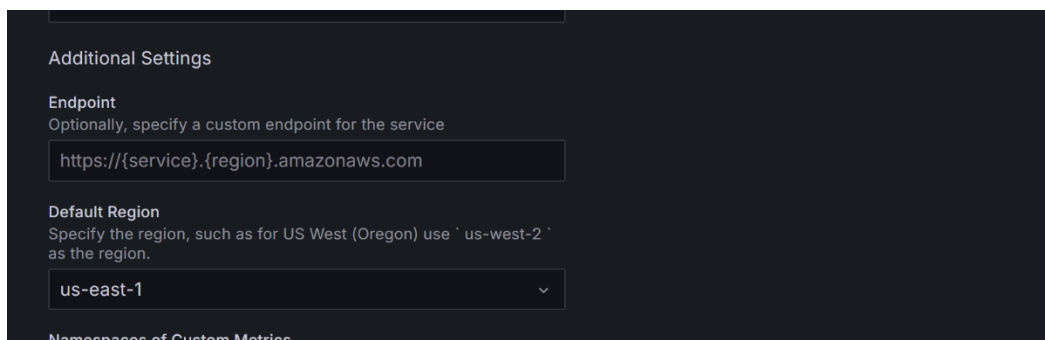
Step 6: Configure CloudWatch Connection

- Enter **Access Key** and **Secret Access Key**.



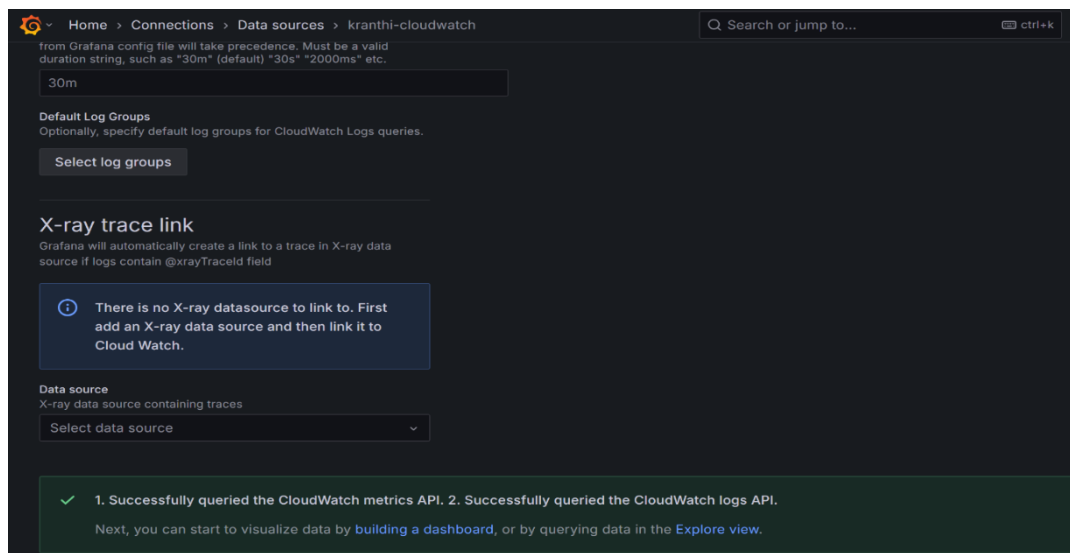
The screenshot shows the 'Connection Details' section of the Grafana configuration interface. At the top, there are fields for 'Name' (kranthi-cloudwatch) and 'Default' (a toggle switch). Below this is the 'Authentication' section. Under 'Authentication Provider', a dropdown menu is set to 'Access & secret key'. The 'Access Key ID' field contains 'AKIA57VDL46HW3MWYC7M'. The 'Secret Access Key' field contains 'dCDxpsHK8qEMA5L959dk5Su+sU+c4zIpyHymmce'. Below this is the 'Assume Role' section, with an 'Assume Role ARN' field containing 'arn:aws:iam:*'. At the bottom is the 'External ID' section, which is currently empty.

- Select **AWS Region** and set **Namespace** to AWS/EC2.



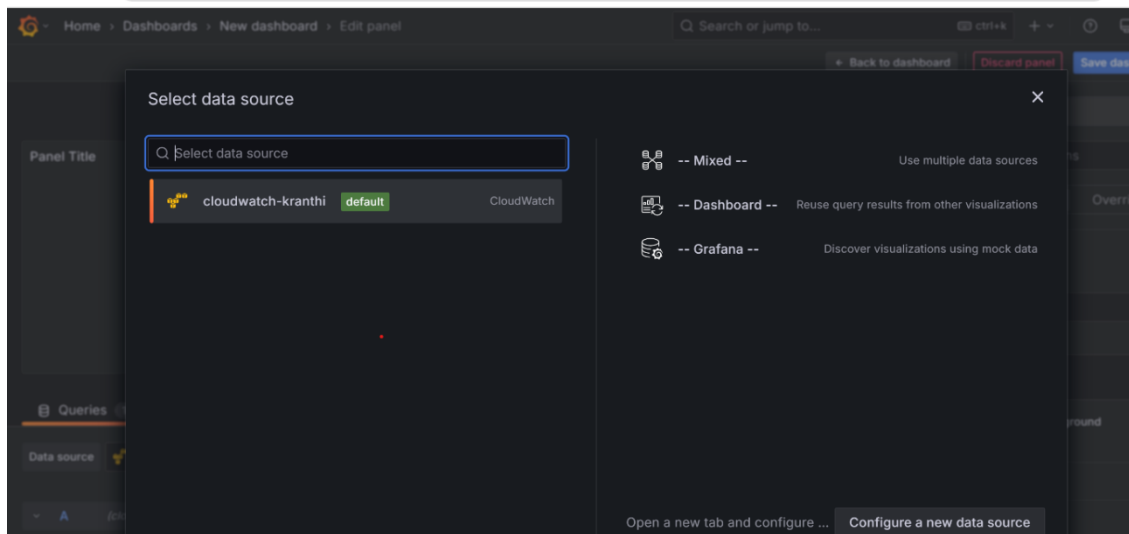
The screenshot shows the 'Additional Settings' section of the Grafana configuration interface. Under the 'Endpoint' section, a text field contains 'https://{service}.{region}.amazonaws.com'. Below this is the 'Default Region' section, with a dropdown menu set to 'us-east-1'. At the bottom, there is a section titled 'Namespaces of Custom Metrics'.

- Click **Test** to verify the connection (should show **Success**).

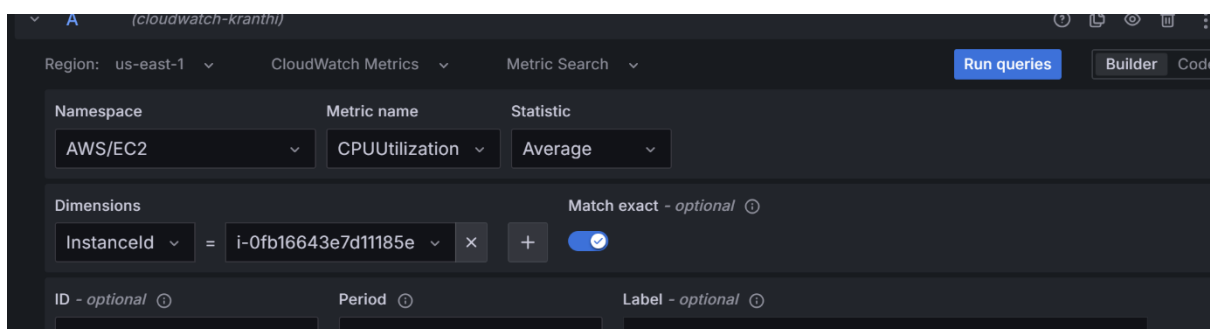
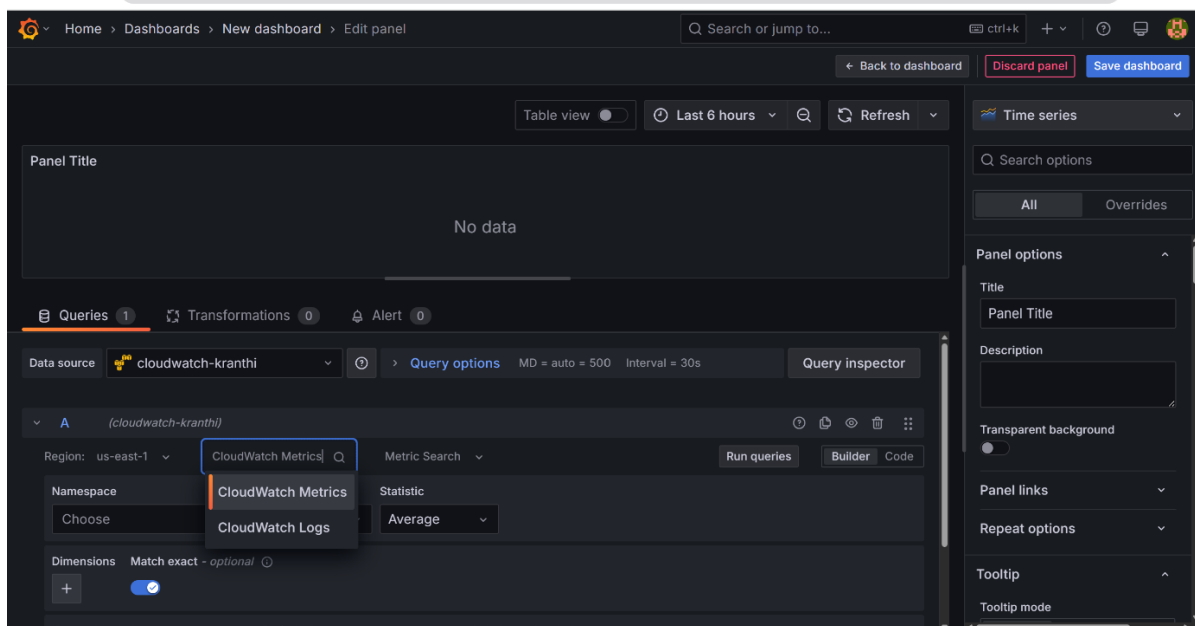


The screenshot shows the 'Data sources' configuration page for 'kranthi-cloudwatch' in Grafana. The breadcrumb trail is 'Home > Connections > Data sources > kranthi-cloudwatch'. The page has a search bar and a 'ctrl+k' shortcut. Below the breadcrumb, there is a section for 'Default Log Groups' with a 'Select log groups' button. The 'X-ray trace link' section contains a message: 'There is no X-ray datasource to link to. First add an X-ray data source and then link it to Cloud Watch.' Below this is a 'Data source' section with a dropdown menu set to 'Select data source'. At the bottom, a green success message states: '1. Successfully queried the CloudWatch metrics API. 2. Successfully queried the CloudWatch logs API. Next, you can start to visualize data by building a dashboard, or by querying data in the Explore view.'

- Go to **Home** → **Dashboards** → **New Dashboard**.
- Select **Add Query** and choose the **CloudWatch** data source.



- Set **Region**, **Namespace (AWS/EC2)**, and select metrics (CPU, Storage, Networking, IOPS, etc.).
- Click **Run Query** to fetch data.



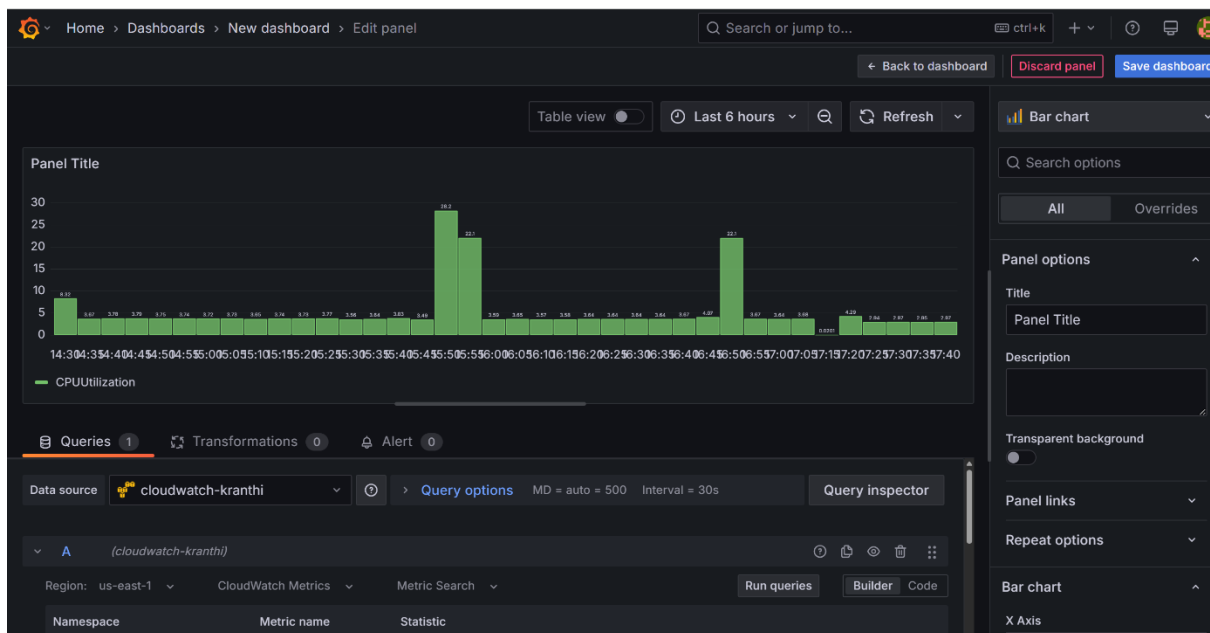
- To see spikes in visualization, run a stress test on EC2: `stress --cpu 4 --timeout 60`

```
Total
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      : 
  Installing      : stress-1.0.7-2.amzn2023.0.1.x86_64
  Running scriptlet: stress-1.0.7-2.amzn2023.0.1.x86_64
  Verifying       : stress-1.0.7-2.amzn2023.0.1.x86_64

Installed:
  stress-1.0.7-2.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-172-31-9-50 ~]$ stress --cpu 4 --timeout 60
stress: info: [28198] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
stress: info: [28198] successful run completed in 60s
[ec2-user@ip-172-31-9-50 ~]$ stress --cpu 4 --timeout 60
stress: info: [28322] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
stress: info: [28322] successful run completed in 60s
[ec2-user@ip-172-31-9-50 ~]$
```

- Visualize Data and you can change visualization by using Bar charts



Conclusion

Grafana provides powerful monitoring and visualization for AWS EC2. Beyond EC2 metrics, you can:

- Monitor **RDS, S3, Lambda, and other AWS services** via CloudWatch.
- Set up **alerts** for resource spikes or failures.
- Integrate with **Prometheus, Loki (logs), and New Relic** for deeper insights.
- Create custom **dashboards** for multi-cloud and hybrid environments.

Grafana helps in proactive monitoring, ensuring better performance and uptime.

Join our group <https://www.linkedin.com/groups/14483928/> **for amazing free resources**